

Phase 3: Requirements Gathering

Project Title: AI-Based In-Cabin Monitoring System

1. Stakeholder Analysis

The system is designed to serve a diverse group of stakeholders, each with specific needs regarding safety, security, and privacy.

Stakeholder	Needs & Expectations
Fleet Management Companies	Require automated logging of driver behavior to ensure compliance with safety protocols and reduce insurance liabilities.
Ride-Sharing Services (e.g., Uber, Careem)	Need assurance of passenger safety and driver attentiveness without compromising cabin privacy (hence the 100% offline requirement).
Armored Vehicles & Cash Transport	High-security verification for drivers (anti-spoofing face recognition) and immediate detection of in-cabin threats (weapons).
Everyday Drivers / Vehicle Owners	Non-intrusive, real-time alerts to prevent accidents caused by fatigue or microsleeps.
Security Teams / Police Patrols	Reliable, evidence-grade timestamped logs and real-time threat evaluation for rapid response.

2. User Stories & Use Cases

User Stories

- **As a fleet manager**, I want the system to log instances of driver distraction (e.g., phone usage) in a local database so that I can review driver performance post-session.
- **As a driver**, I want the system to sound a clear audio alert if I begin to fall asleep ($EAR < 0.24$ for $> 1.5s$) so that I can pull over safely.
- **As a ride-share passenger**, I want the system to actively detect weapons or violent behavior so that my safety is monitored during the trip.
- **As a vehicle owner**, I want the vehicle to verify my identity using face recognition before starting a session to prevent unauthorized access or theft.

Core Use Cases

1. **Driver Authentication Session:** The camera scans the driver's face. The system runs anti-spoofing checks (blur, texture, edge). If passed, ArcFace extracts the embedding and matches it against the local `users_db.json`.
2. **Fatigue & Microsleep Prevention:** The system continuously monitors the driver using MediaPipe and YOLO11n. If PERCLOS exceeds 70% or 3 yawns are detected in 60 seconds, a warning is triggered.
3. **Threat Detection & Logging:** YOLOv8n scans for weapons while VD-MIL analyzes temporal frames for violent motion. If a threat is detected, the dashboard updates the threat level (Watch/Warning/Critical) and logs the event to SQLite.

3. Functional Requirements

The system must perform the following explicit actions:

- **F-REQ-01:** The system shall authenticate users via ArcFace with an anti-spoofing pipeline prior to session initiation.
- **F-REQ-02:** The system shall detect specific distraction objects in the driver's view, explicitly: Phones, Cigarettes, Vapes, and Bottles.
- **F-REQ-03:** The system shall calculate Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) to determine drowsiness and yawning states.
- **F-REQ-04:** The system shall process 8-frame clips through a MoViNet A0 + MIL architecture to assign a violence probability score.
- **F-REQ-05:** The system shall output a live MJPEG video stream to a Flask-based web dashboard.

- **F-REQ-06:** The system shall trigger visual (color-coded HUD banners) and audio (880 Hz beep) alerts based on threat severity.
- **F-REQ-07:** The system shall automatically log all alerts and events, including timestamps and durations, to a local SQLite database.

4. Non-Functional Requirements

These define the quality, performance, and constraints of the system.

- **Performance & Latency:** End-to-end inference latency must remain under 50ms per frame. The UI dashboard must poll updates at a minimum of 2Hz (every 500ms). Threat evaluation must trigger within ≤ 2 seconds of the event.
- **Hardware Constraints:** The entire pipeline must run smoothly at ~30 FPS on a single consumer-grade NVIDIA GTX 1650 or 1660 Super GPU using CUDA 11.8/12.1. VRAM usage during inference must not exceed 4GB.
- **Privacy & Security:** The system must operate 100% offline (Edge Computing) without transmitting live video feeds or biometric data to external cloud servers.
- **Usability:** The web dashboard must utilize a dark theme to reduce driver eye strain during night operations and ensure threat colors (Red, Orange, Yellow) remain highly visible.
- **Maintainability:** The codebase must be modular, adhering to Python 3.11.5 standards, with clear separation between the Flask server thread and the background detection threads to prevent race conditions.